

theory of Computation

Relation Machine will do computations on the basis of Theory

Definition: Theory of Computation is branch of computer science that deals with how efficiently problem can be solved on a model of computation using an algorithm.

Theoretical computations is built upon two foundational elements: (i) Principles (ii) Concepts

(i) Principles: The underlying rules, mathematical logic, formal laws and proofs. (e.g. set theory, formal languages)

(ii) Concepts: Abstract ideas and models (e.g. state transition, decidability) etc.

Every Computational machine is defined by four core characteristics: (i) Compatibility (ii) Limitations (iii) Complexity (iv) Performance.

(i) Compatibility: How different machine models equate to or convert into one another.

(ii) Limitations: What a specific machine model can or cannot compute/recognize.

(iii) Complexity: The amount of resources (time and space/memory) a machine consumes for an input size n .

(iv) Performance: How efficiently or conveniently a machine executes computation.

Application of TOC

(i) It plays an important role in compiler design.

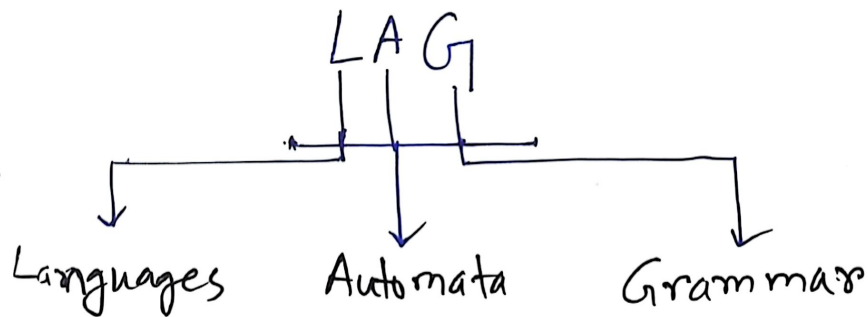
(ii) In switching theory and design and analysis of digital circuits automata theory is applied.

(iii) Design and Analysis of Complex hardware and software system.

iv) To improve ~~connections~~ness of the programm automata theory is used.

v) To design finite state machines such as ~~more~~ moore and mealy machines

Pillars of TOC:



Language

to make language we need →

- Symbols
- Alphabets
- Strings

Symbols: Digits, letters
0, 1 a-z

Def: Basic building block of a language or smallest unit of a language are called Symbols

Alphabets: Finite set of symbols are called Alphabets. It is denoted by $(\Sigma) \rightarrow$ sigma

For example: $\Sigma = \{a, b\}$, $\Sigma = \{0, 1\}$

Strings: Finite/Infinite sequence of symbols are called Strings. It is denoted by $(w) \rightarrow$ omega

Examples: $\Sigma = \{a, b\}$ and string length = 2, strings?

$\therefore w = \{aa, bb, ab, ba\}$ [finite sequence]

formula $\rightarrow$ ~~(Σ)~~ $\left[\begin{array}{c} \text{string length} \\ \Sigma \end{array} \right]$

length of string denoted as $|w|$

Ques: $w = \{a b a a b a\}$ sequence = 1.

$|w| = ?$ $\rightarrow$ length = 6

Language: Collection of all possible strings are called Language. For example: ~~[All C-programms]~~ are strings. And If it is valid then that is Language]

Power of Sigma (Σ) (alphabets)

Lecture-3
27.07.2026

$\Sigma^0(\Sigma)$ epsilon

formula
 $\Sigma^{\text{str-length}}$

$$\Sigma^1 = \{a, b\} / \{0, 1\}$$

$$\Sigma^2 = \{aa, bb, ab, ba\}$$

$$\Sigma^3 = \{ \dots \}$$

$$\Sigma^K = \{ \dots \}$$

mother of all strings

⇒ If sigma is an alphabet, we can express set of all strings of certain length from that alphabet by using exponential notation. It is denoted by Σ^K K is the set of strings length

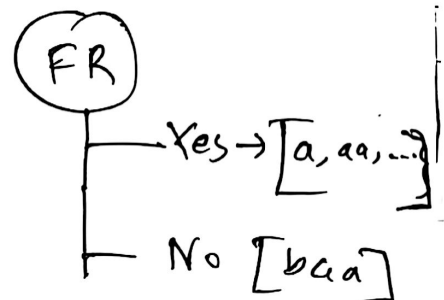
$$\Sigma = \{a, b\}$$

$$L = \{w \mid w, \text{ where each string starts with } a\}$$

$$= \{a, aa, aaa, ab, \dots\}$$

↓ Infinite Lang.

Finite Representation
Finite Automata



Automata theory and Language

It deals with definition and properties of various mathematical models of computation.

Definition: Automata is a self operating system or machine that performs predefined sequences of operation automatically.

Mathematical Models: (i) Finite Automata (ii) Context Free Grammar
(iii) Turing Machine.

Types of Automata:

- (i) Finite Automata (FA)
- (ii) Push down Automata (PDA)
- (iii) Linear Bound " (LBA)

Finite Automata: F.A. is an abstract computing device. It is a mathematical model of a system with discrete inputs, outputs, states and set of transitions from state to state that occurs on input symbols from alphabets.

representation: (i) Graphical { transition diagram or transition table }
(ii) tabular { transition table }
(iii) Mathematical { transition function or mapping function }

Formal Definition of FA

Lecture-4

03.08.2026

A finite Automata is a set of 5 tuples. They are;

$Q, q_0, F, \Sigma, \delta$, where,

Q = set of states, is a finite called states.

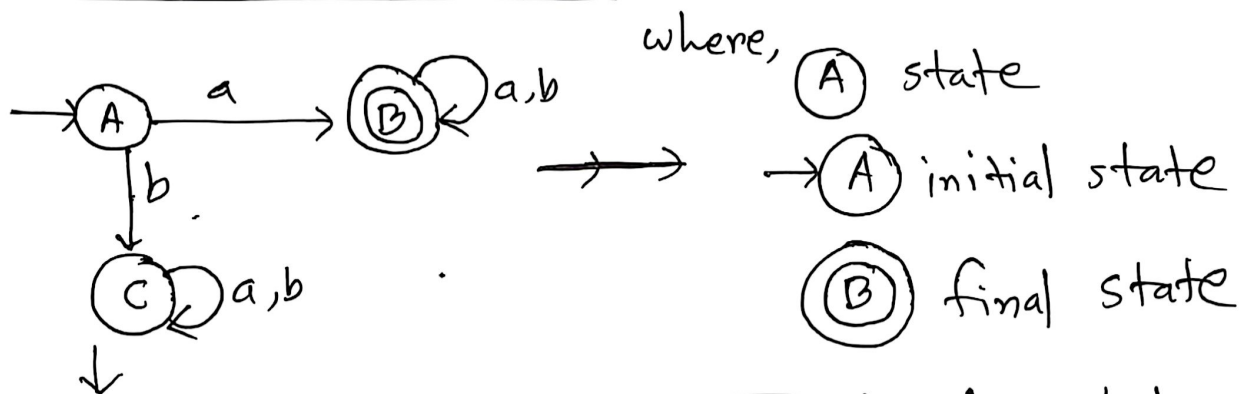
q_0 = initial state

F = Final state

Σ = Alphabet, is a finite set called alphabets.

δ = Transition function.

State transition diagram:



Q = set of states = $\{A, B, C\}$

q_0 = initial state = A

F = final state = B

$\Sigma = \{a, b\}$ = alphabets

δ = Transition function = $Q \times \Sigma \rightarrow Q$

$\textcircled{A} \Rightarrow$ trap state
Conditionally 2026

states	a	b
$\rightarrow A$	B	C
$*B$	B	B
C	C	C

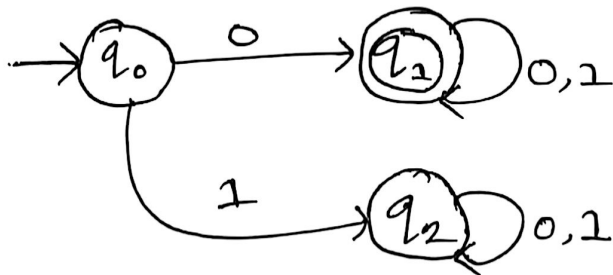
Q. - 10 उत्तर।

(i) formal definition of FA. {उपर्युक्त पर लिखिए}

(ii) Figure उत्तर formal def लिखिए।

(iii) यदि (ii) उत्तर उत्तर value 3 लिखिए।

Example-1



Define Finite Automata (formal definition)?

Solⁿ: A finite Automata is a set of 5 Tuples. They are:

$Q, q_0, F, \Sigma, \delta$ where;

Q = set of states = $\{q_0, q_1, q_2\}$

q_0 = initial state = q_0

F = final state = q_1

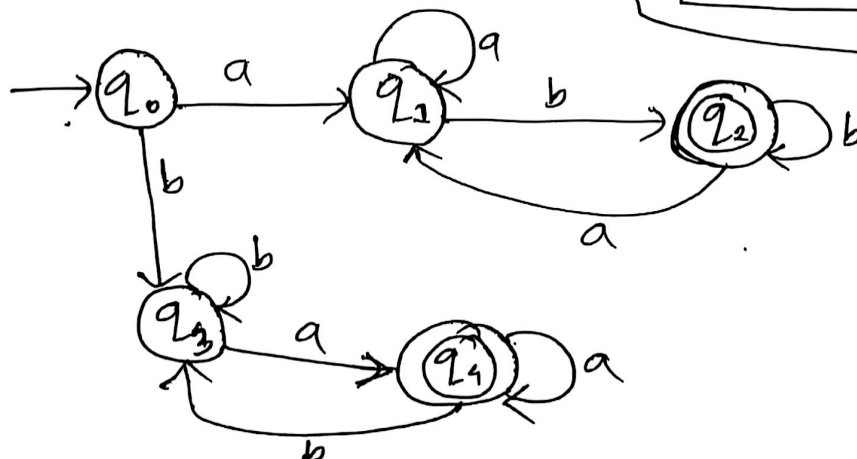
$\Sigma = \{0, 1\}$ = alphabets

δ = Transition function $\rightarrow$

$= Q \times \Sigma = Q$

states	0	1
$\rightarrow q_0$	q_1	q_2
$*q_1$	q_1	q_1
q_2	q_2	q_2

Example-2



Answer: A finite Automata is a set of 5 tuples. They are:

$Q = \text{set of states} = \{q_0, q_1, q_2, q_3, q_4\}$

$q_0 = \text{initial state} = q_0$

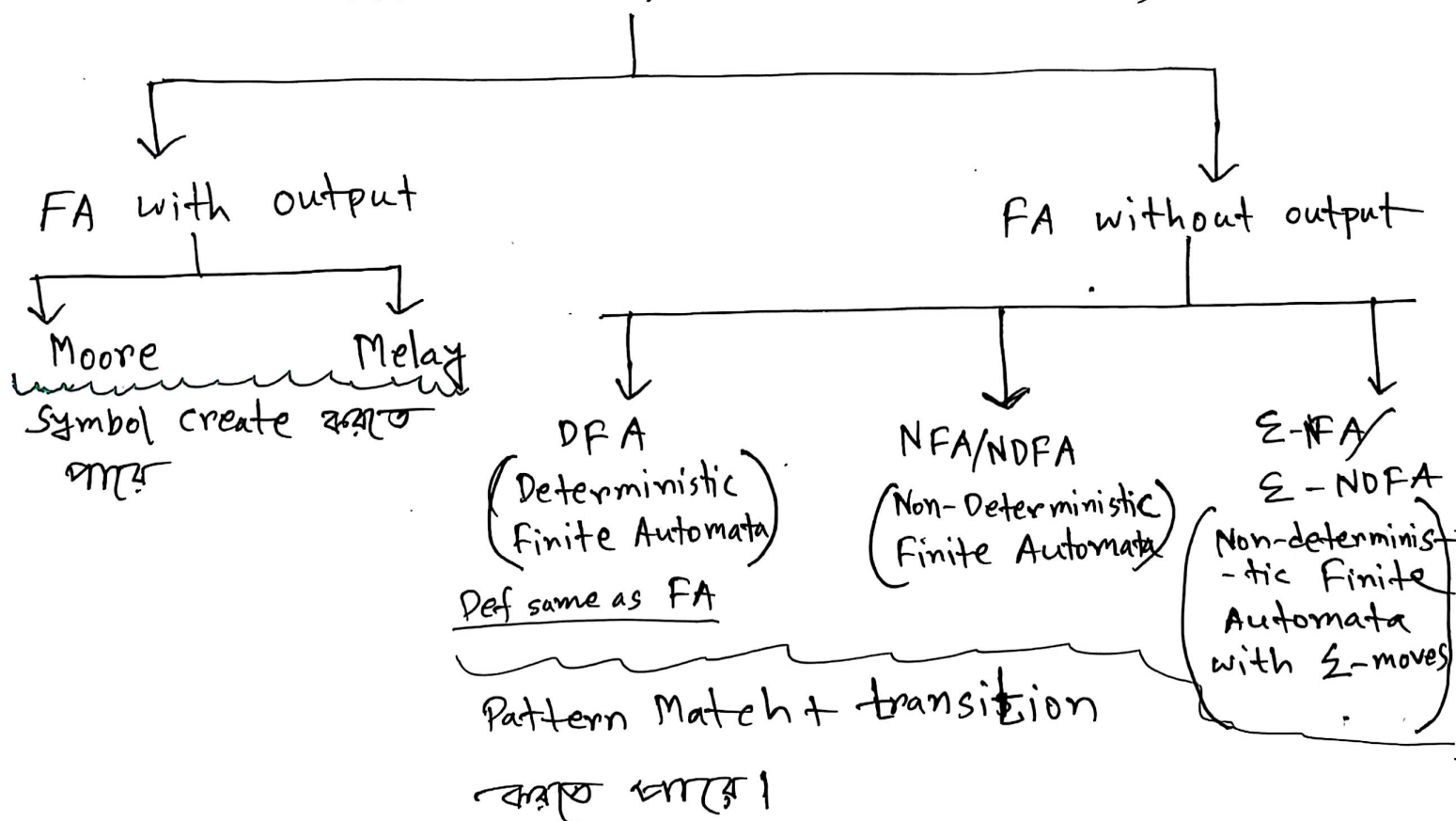
$F = \text{final state} = \{q_2, q_4\}$

$\Sigma = \{a, b\} = \text{alphabets}$

$\delta = \text{transition function} \rightarrow$
 $= Q \times \Sigma = Q$

$Q \times \Sigma$	a	b
$\rightarrow q_0$	q_1	q_3
q_1	q_1	q_2
$*q_2$	q_2	q_2
q_3	q_4	q_3
$*q_4$	q_4	q_3

Classification of Finite Automata (FA)



Deterministic: Deterministic means that the outcome is 100% fixed by the initial conditions, so the exact same cause will always lead to the exact same effect with no randomness. Example:

⇒ Consider a ^{math} function $f(n) = n + 5$. It is deterministic because if $n = 3$ $f(3)$ will always equal to 8. It will never result 7 or 9.

⇒ Consider an array $[4, 2, 9, 1]$. After sorting the number using merge sort it will always result $[1, 2, 4, 9]$ and will always carry this.

Deterministic vs Non-Deterministic

<u>Concept</u>	<u>Deterministic</u>	<u>Non-Deterministic</u>
Path	Single, fixed linear path	Multiple potential path or possibilities
Behavior	Predictable (100% certainty)	Uncertain, random or multi-choice
Reproducibility	Perfect - running it twice still gives the exact same outcome	Outcome or steps may vary across runs.

Criteria of DFA

- i) প্রত্যেকটা states প্রাক অবস্থান symbol নিয়ম কাজে নিশ্চিত করা ensure করতে হবে।
- ii) অবস্থান state প্রাক প্রাক্কারণ symbol মুক্তি প্রকারণ হে কাজে করা
- iii) Conditionally trap state প্রাক্কারণ হে পারে।
- iv) initial state always একটা হবে কিন্তু final state এক/একটি হবে।

DFA (Deterministic Finite Automata)

Lecture-5
06.08.2026

Formal

Def: A Deterministic Finite Automaton (DFA) is a set of 5 tuples: They are $Q, q_0, F, \Sigma, \delta$, where:

Q = set of states, it is a finite called states.

q_0 = initial state

F = final state

Σ = Alphabet, is a finite set called alphabets.

δ = Transition function = $Q \times \Sigma$
= Q

Construct Criteria of DFA: উল্লিখিত নীতি ॥

Examples of DFA

Category: where each string starts with 'a'.

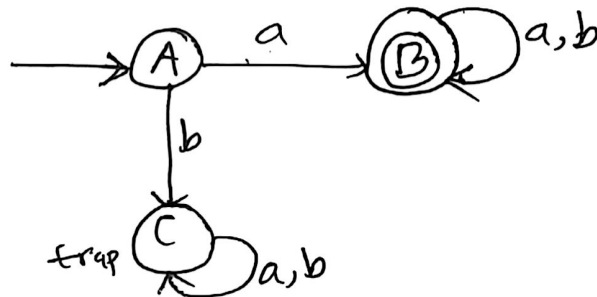
Example-1: Construct a DFA which accepts set of all strings over $\Sigma = \{a, b\}$ where each string starts with 'a'.

* remaining symbol ka diagram - $\rightarrow$ $\rightarrow$ $\rightarrow$ sequence:

- ① Self loop ② Back state ③ Trap state.

Soln: $L = \{a, aa, aaa, ab, aba, \dots\}$

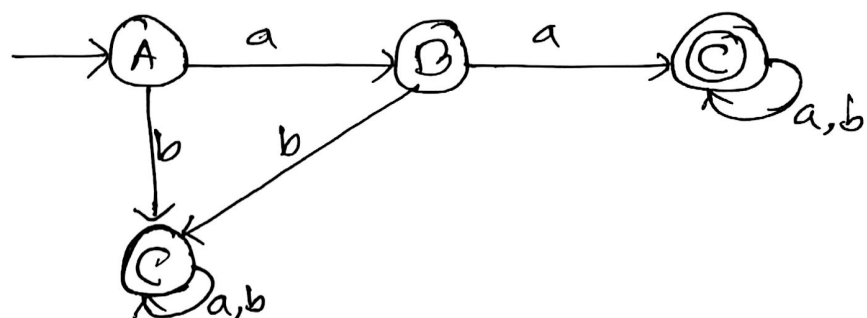
state transition:



Example-2: $\dots$, where each string starts with 'aa'.

Soln: $L = \{aa, aqa, aqb, aqbb, aqba, \dots\}$

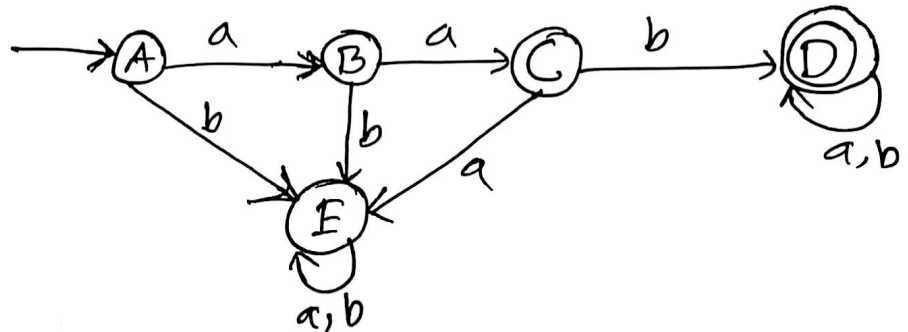
state transition:



Example-3: each string start with "aab". -

Solⁿ: $L = \{ aab, aaba, aabba, aabb, \dots \}$

State transition:

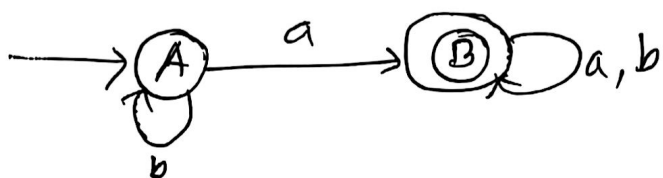


Category: where each string contains "a" as sub-string

Example-4: Construct a DFA which accepts set of all strings over $\Sigma = \{a, b\}$ where each string contains 'a' as sub-string

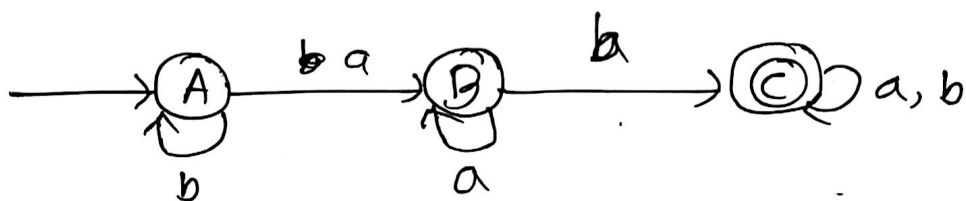
Lecture-6
10.08.2026

Solⁿ: $L = \{ a, aa, aaa, ba, ab, bab, \dots \}$



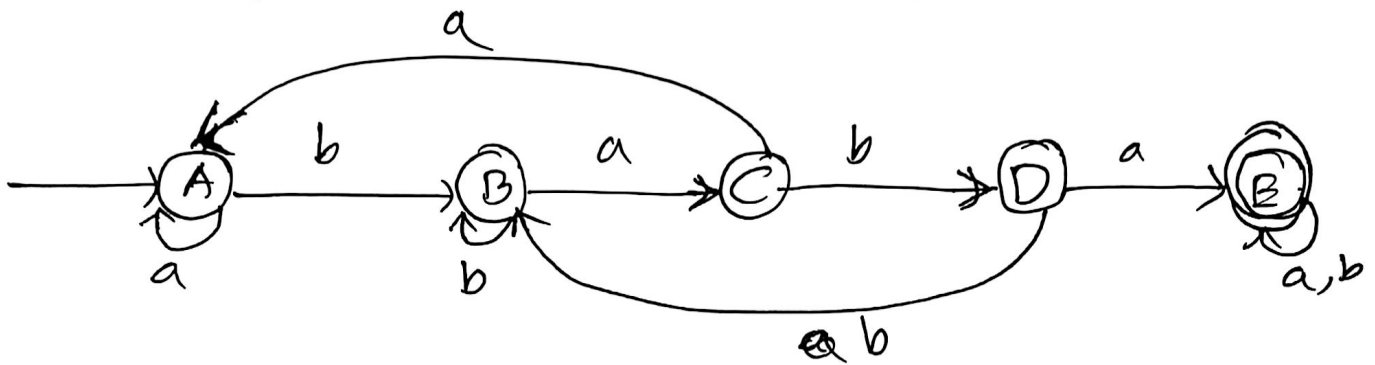
Examples:-5 contains 'ab' as sub-string

Solⁿ: $L = \{ ab, aba, aaba, bbab, abba, \dots \}$



Example-6: ... contains w 'baba' as sub-string.

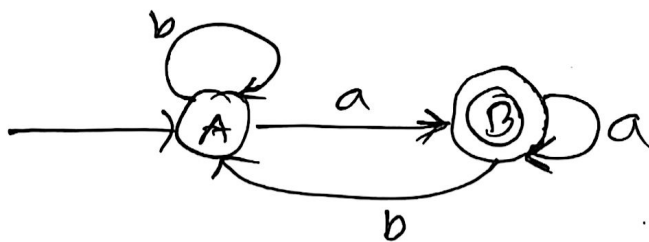
Soln: $L = \{ \text{baba, ababa, ababab, abababb, ...} \}$
 babbaba



Category: where each string ends with '...'

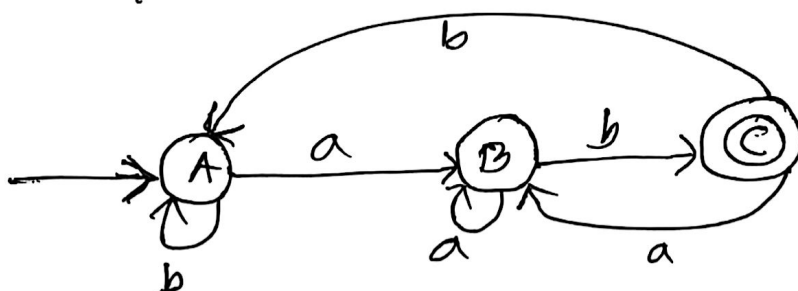
Example-7: Construct a DFA which accepts set of all strings over $\Sigma = \{a, b\}$ where each string ~~start~~ ends with 'a'.

Soln: $L = \{ a, aa, aaa, ba, aba, \dots \}$



Example-8: ... ends with 'ab'

Soln: $L = \{ ab, aab, abbab, abab, \dots \}$



Category-4:

Lecture-7

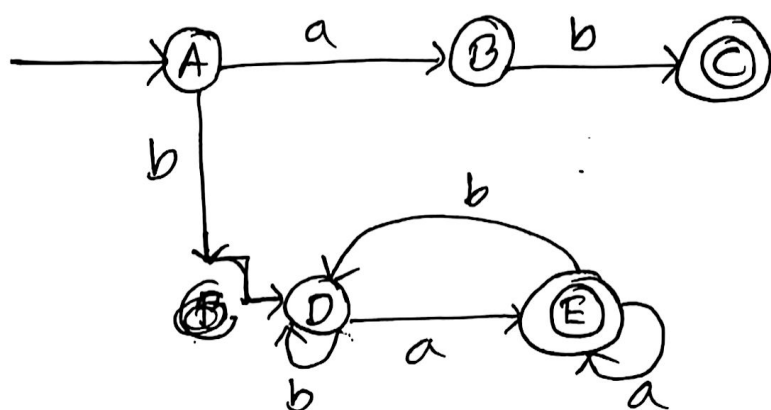
13.08.2026

$$\Sigma = \{a, b\}$$

Example-9: ... where $L = \{ w/w, \text{ where each string starts and ends with different symbol} \}$

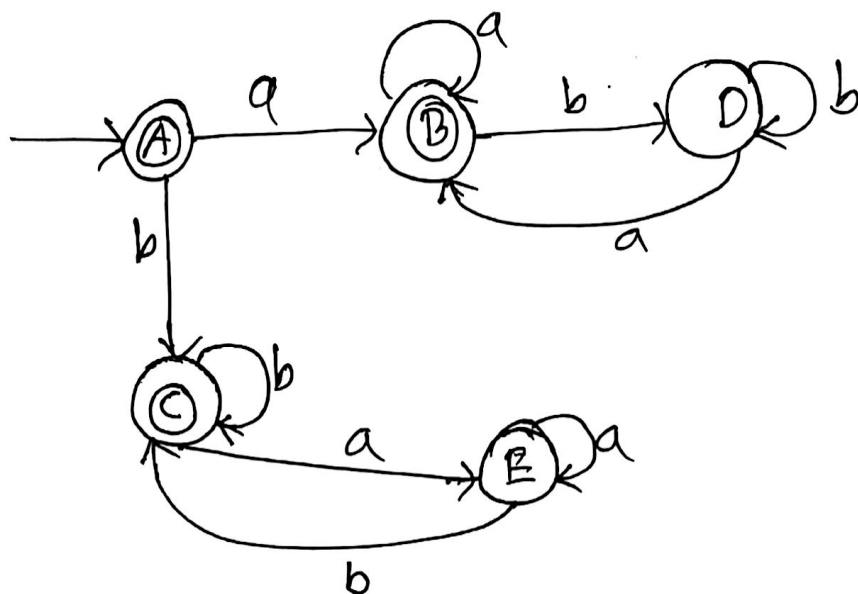
Soln:

$$L = \{ ab, ba, abb, abbb, baa, baab, \dots \}$$



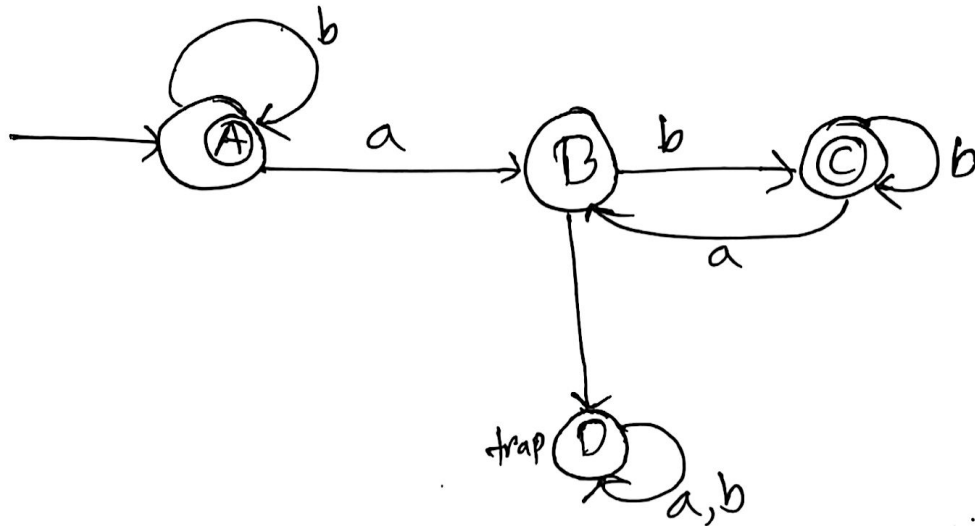
Example-10: ... $\Sigma = \{a, b\}$ where $L = \{ w/w, \text{ where each string starts and ends with same symbol} \}$

Soln: $L = \{ \epsilon, a, b, aa, bb, abba, baab, \dots \}$



Example-11: Every 'a' in w is followed by 'b'.

Solⁿ: $L = \{ \epsilon, b, bbb, ab, abbab, \dots \}$



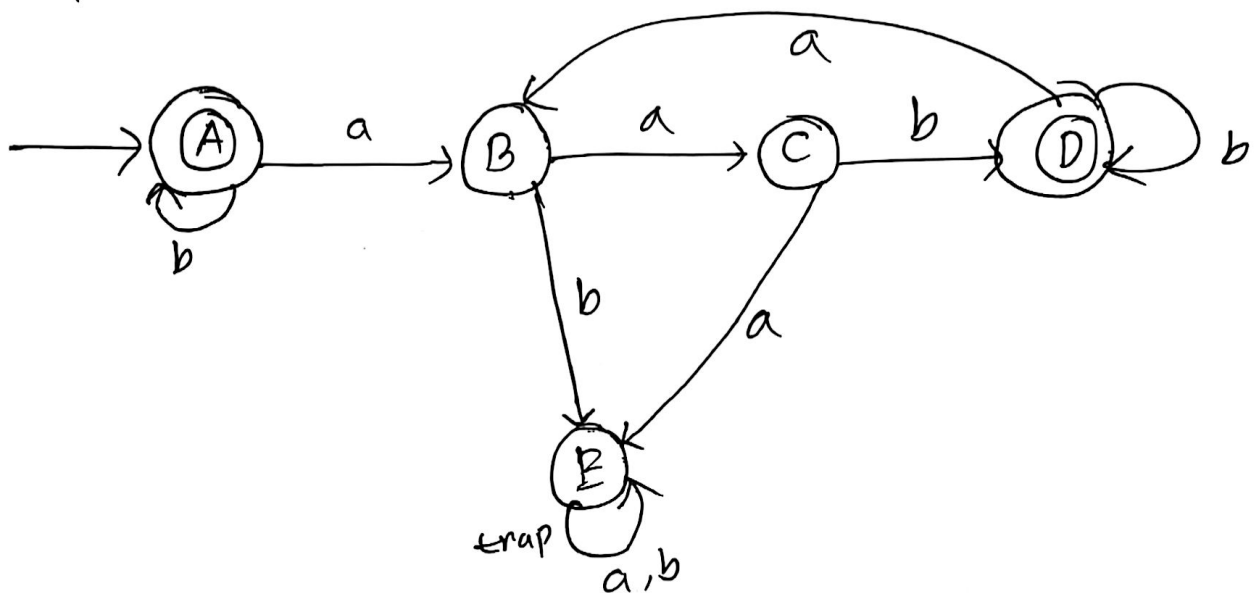
Example-12

Lecture-8

17-08-2026

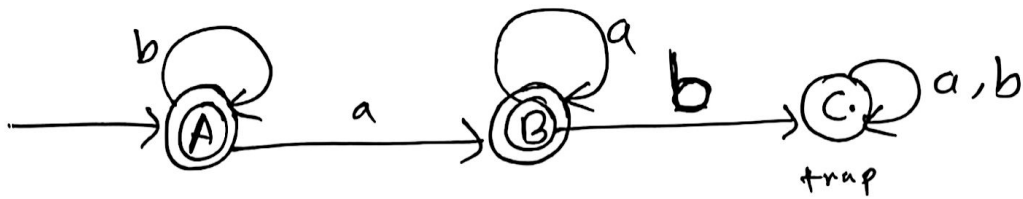
'a' is in w is followed by 'ab'

$L = \{ \epsilon, b, bb, bbb, aab, aab, aabbb, bbbbaab, \dots \}$



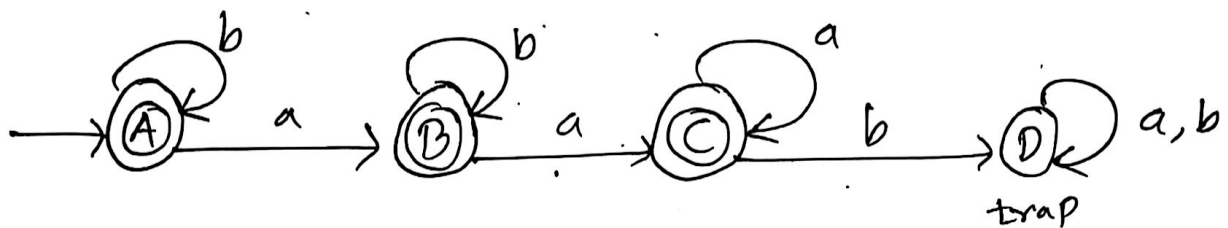
Example - 13: 'a' in w is never followed by 'b'.

$$L = \{ \epsilon, a, b, ba, aa, bb, aaa, bbb, baa, \dots \}$$



Example - 14: 'a' in w is never followed by 'ab'.

$$L = \{ \epsilon, a, b, bbb, ab, bbbabb, abaa, bab, \dots \}$$

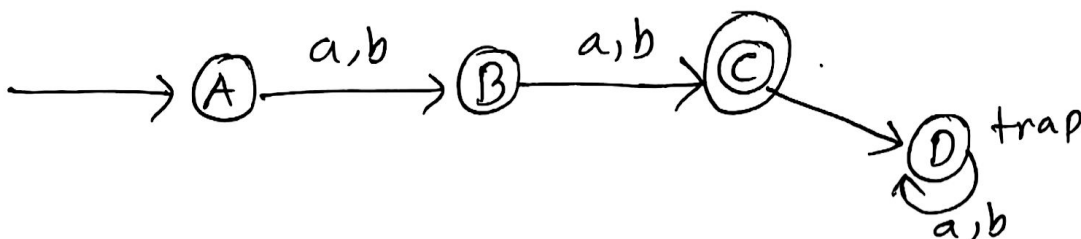


Category: length of string

$$2^2 = 4 \rightarrow \text{combination}$$

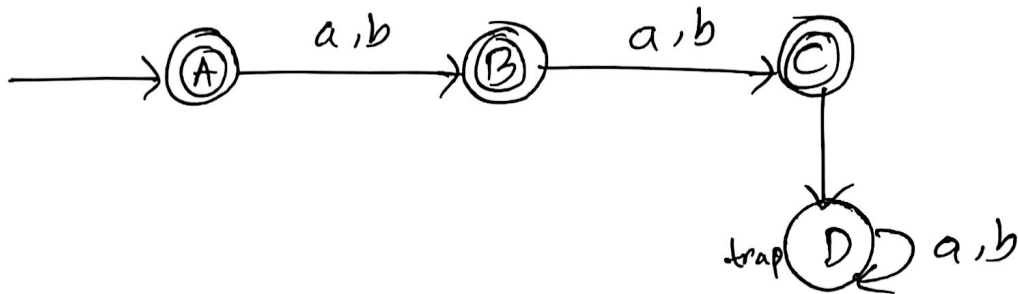
Example-1: length is 2

$$L = \{ aa, ab, bb, ba \} \leftarrow \text{finite language}$$



Example-2: Length is at most 2 } ~~ଅର୍ଥାତ୍ 2~~
~~ଅର୍ଥାତ୍ ϵ~~

$$L = \{ \epsilon, a, b, aa, bb, ab, ba, \}$$



Example-3: Length atleast 2:

$$L = \{ aa, bb, ab, ba, aaa, bbb, aba, abb \}$$



Why DFA minimization?

- i) reduces number of states.
- ii) time, memory and performance increase.

Definition: DFA Minimization is the process of transforming a given Deterministic Finite Automaton (DFA) into an equivalent DFA that recognizes the exact same regular language.

Why?

i) Reduces the number of states: Eliminates unreachable states from the start state and ~~merges~~ merges equivalent states into single equivalence classes producing minimal possible state.

ii) Time, Memory and Performance: Smaller DFAs result in ~~drastically~~ reduced states use less memory space to store transition rules and run more efficiently. Because there are fewer states and transitions to look up, it speeds up processing when checking input strings.

iii) Language Equivalence and Testing: It creates a standard, unique version of a machine for any regular language, making it simple to check if two different machines accept the exact same language.

Procedure for DFA minimize:

- i) Remove all the states those are not reachable from initial states.
- ii) Draw state transition table
- iii) Mark initial and final state
- iv) Equivalent.

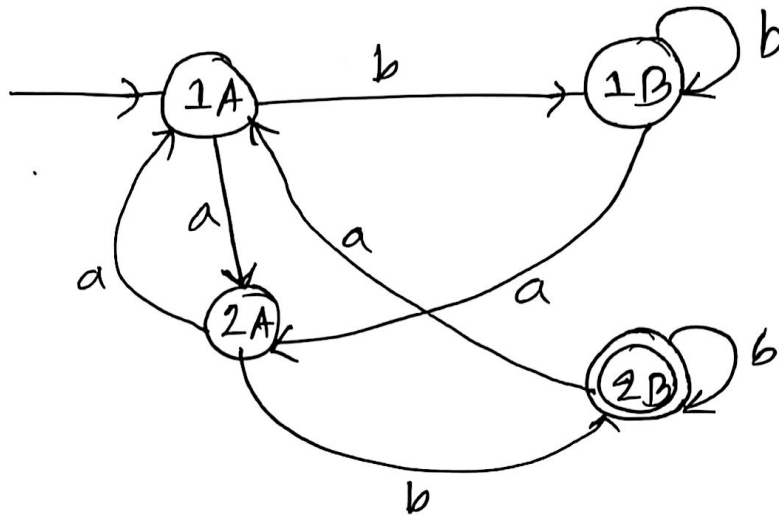
যদি দুটি স্টেট একই ভাবে run করে তবে equivalent run হবে।

if (P, Q) are two states then:

i) if $\delta(P, w) \in M$ then $\delta(Q, w) \in M$

ii) if $\delta(P, w) \notin M$ then $\delta(Q, w) \notin M$

Example-1



Minimization:

1. doesn't need.

2,3. step-2 table

step-3 "

	a	b
→1A	2A	1B
1B	2A	1B
2A	1A	2B
*2B	1A	2B

① 0 equivalent
 $\hookrightarrow = [\text{set of final states}] \times [\text{set of non-final states}]$

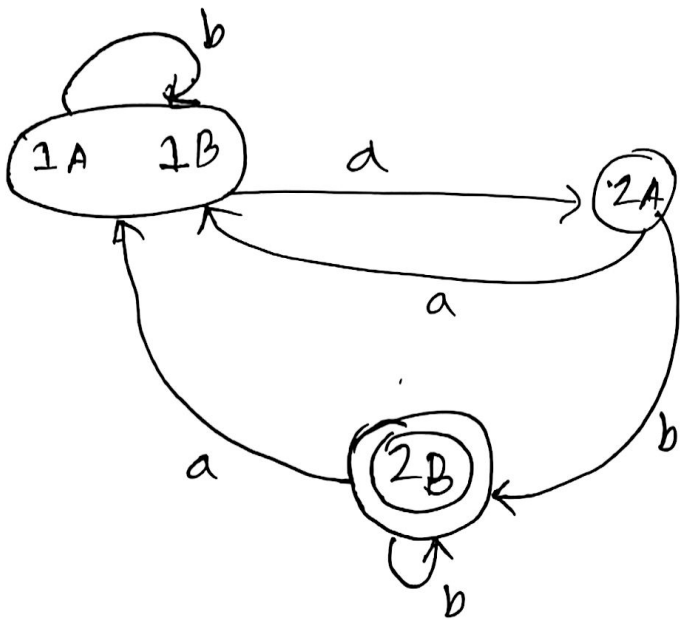
0 equivalent
 $\hookrightarrow = [2B] [1A \ 1B \ 2A]$

1 equivalent
 $\hookrightarrow = [2B] [1A \ 1B] [2A]$

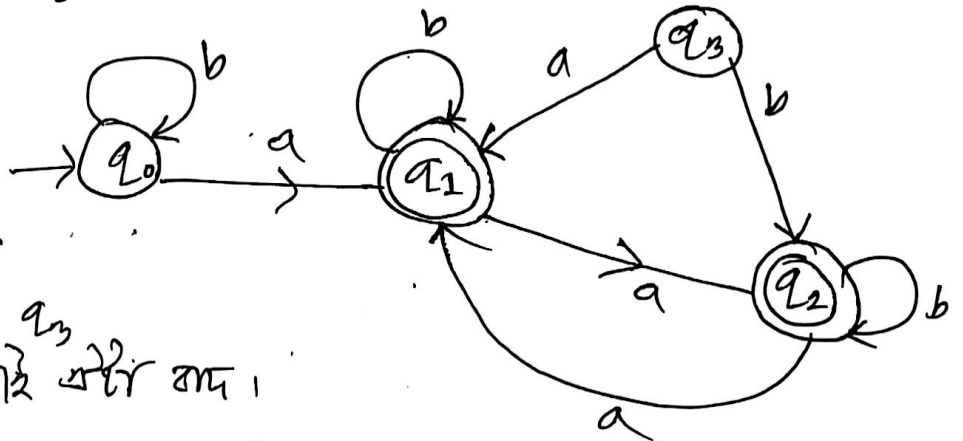
2 equivalent
 $\hookrightarrow = [2A] [2B] [1A \ 1B]$

② $1A \xrightarrow{a} 2A$
 $1B \xrightarrow{a} 2A$

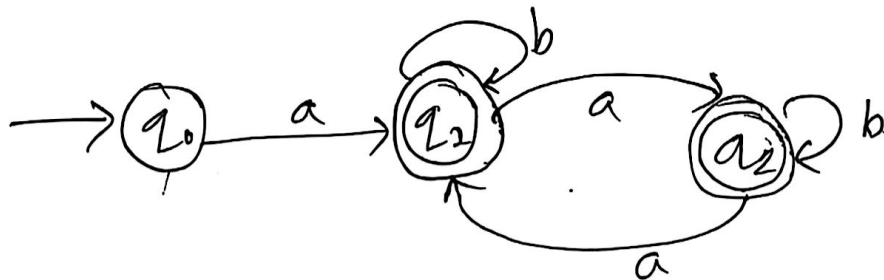
$1A \xrightarrow{b} 1B$
 $2B \xrightarrow{b} 1B$



Example-2:



step-1: initial (ଆରମ୍ଭ) q_3
 ଓ ଶେଷ (ଶେଷ) q_1 ଓ q_2 ରେ ଯିବା ପାଇଁ ଗଢ଼ାଯାଏ ।



Step-2 } table and initialization:
 step-3 }

	a	b
$\rightarrow q_0$	q_1	q_2
$*q_1$	q_1	q_1
$*q_2$	q_1	q_2

Step-4: $\circ$ equivalent = $[q_1 \ q_2] \ [q_0]$

1 equivalent = $[q_0] \ [q_1 \ q_2]$

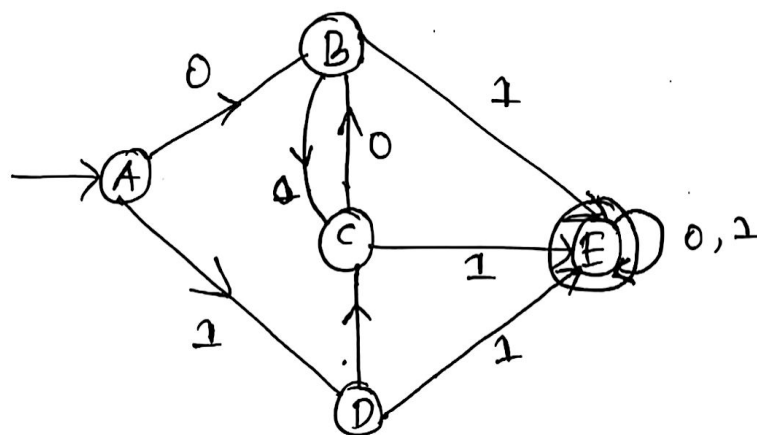
2 equivalent = $[q_0] \ [q_1 \ q_2]$

(Ans)

i) $\begin{matrix} q_1 \\ q_2 \end{matrix} \xrightarrow{a} \begin{matrix} q_2 \\ q_1 \end{matrix}$ $\begin{matrix} q_1 \\ q_2 \end{matrix} \xrightarrow{b} \begin{matrix} q_1 \\ q_2 \end{matrix}$

Example-3

Lecture-10
27.08.2026



Step-1: all vertex reachable.

Step-2: } Table:
Step-3: }

initialize

	0	1
$\rightarrow A$	B	D
B	C	E
C	B	E
D	C	E
*E	E	E

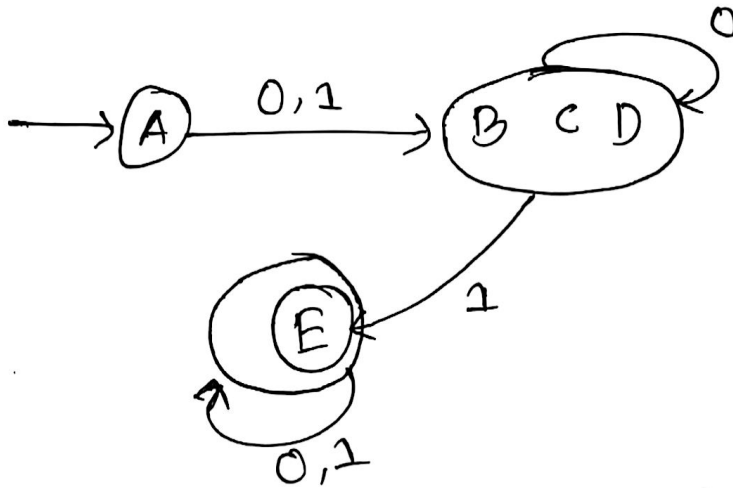
Step-4: $\circ$ equivalent = $\left[\begin{matrix} \text{set of final} \\ \text{states} \end{matrix} \right] \times \left[\begin{matrix} \text{set of non-final} \\ \text{states} \end{matrix} \right]$

= $[E] \ [A \ B \ C \ D]$

1 equivalent = $[E] [A] [\overline{B}CD]$

2 equivalent = $[E] [A] [\overline{B} c D]$

Final diagram:



testing

i) $\begin{matrix} A & 0 & B \\ B & & C \end{matrix} \quad \begin{matrix} A & 1 & D \\ B & & E \end{matrix}$

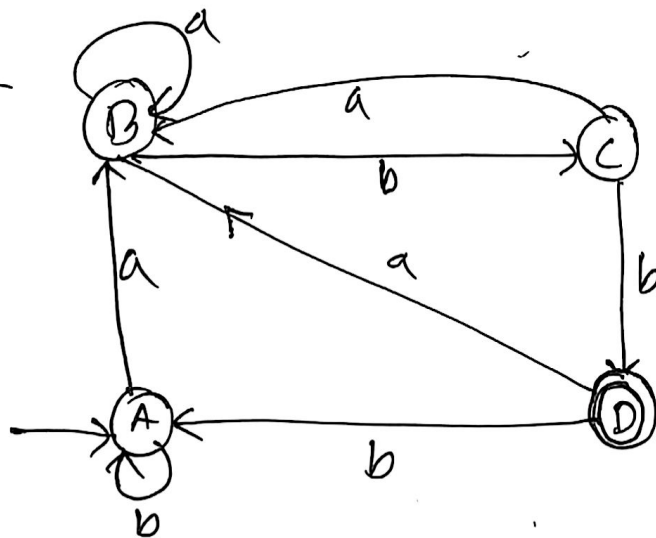
ii) $\begin{matrix} A & 0 & B \\ C & & B \end{matrix} \quad \begin{matrix} A & 1 & D \\ C & & E \end{matrix}$

iii) $\begin{matrix} B & 0 & C \\ C & & B \end{matrix} \quad \begin{matrix} B & 1 & E \\ C & & E \end{matrix}$

iv) $\begin{matrix} A & 0 & B \\ D & & C \end{matrix} \quad \begin{matrix} A & 1 & D \\ D & & E \end{matrix}$

v) $\begin{matrix} B & 0 & C \\ D & & C \end{matrix} \quad \begin{matrix} B & 1 & E \\ D & & E \end{matrix}$

Example-4



Step-1: all vertex reachable

Step-2: table

Step-3: initialization

	a	b
→ A	B	A
B	B	C
C	B	D
* D	B	A

step-4: 0 equivalent = [set of final states] x [set of non-final states]

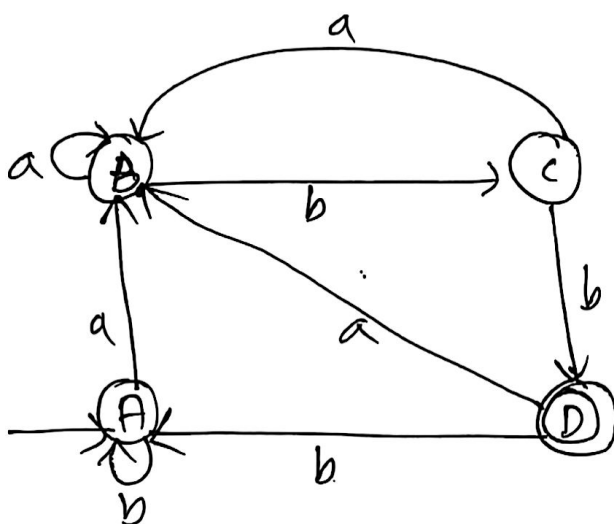
$$= [D] [A B C]$$

1 equivalent = $[D] [A B] [C]$ | i) $\begin{matrix} A & \xrightarrow{a} & B \\ B & & B \end{matrix}$ $\begin{matrix} A & \xrightarrow{b} & A \\ B & & C \end{matrix}$

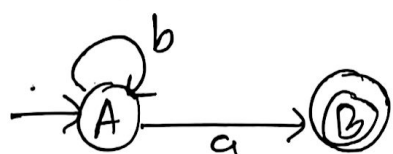
2 equivalent = $[D] [C] [A] [B]$ | ii) $\begin{matrix} B & \xrightarrow{a} & B \\ C & & B \end{matrix}$ $\begin{matrix} B & \xrightarrow{b} & C \\ C & & D \end{matrix}$

3 equivalent = $[A] [B] [C] [D]$

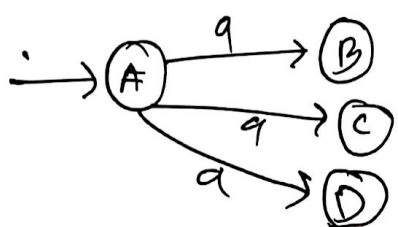
final diagram;



NFA/NDFA



} DFA
must loop fill-up कराउ शक



} NFA
Loop दिअ fill-up ना कराउ शक
एक symbol दिअ काहू ना कराउ शक।

Definition: The finite automata called ~~A~~ NFA when there exists many paths for specific input or output from the current state to the next state. It is easy to construct NFA than DFA for a given regular language. Every NFA is not DFA, but every NFA can be translated in DFA.

DFA & NFA ~~are~~ differences

DFA	NFA
1) Only one path for specific input	1) Many path for specific input.
2) Exactly 1 state to next Move without reading input(ϵ) not allowed	2) Move without reading input(ϵ) allowed.
3) Execution path single, predictable linear path	3) Multiple branching paths (a tree of possibilities)
4) Strictly follow predefined rules	4) Explores multiple paths simultaneously.
5) A string x is accepted if and only if its single designated path finishes in an accepting state.	5) A string x is accepted if at least one computational branch reaches an accepting state.

Formal Definition: A Non-Deterministic Finite Automaton (NDFA) is a set of 5 Tuples: They are $Q, q_0, F, \Sigma, \delta$

where: Q = set of states; is a finite set called states.

q_0 = initial state

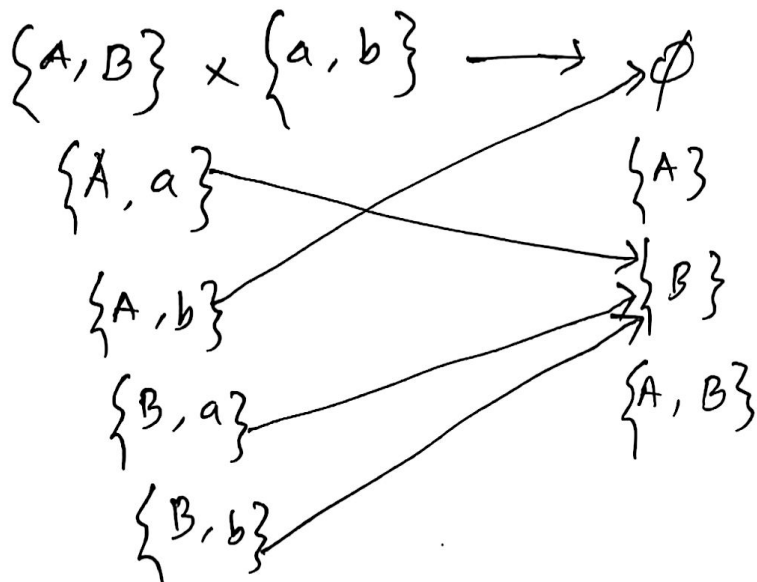
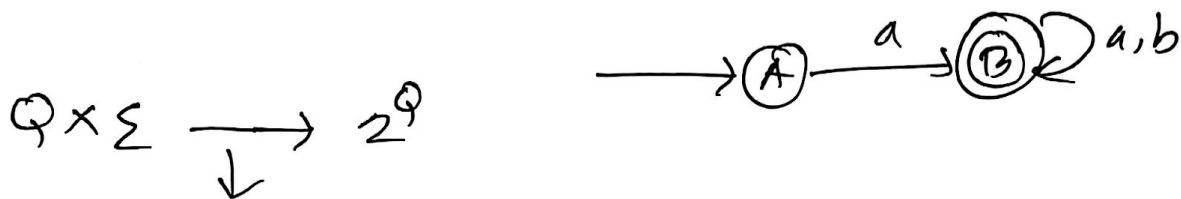
F = final state

Σ = Alphabet, is a finite set called alphabets.

δ = Transition state

$\hookrightarrow Q \times \Sigma \rightarrow 2^Q$ [subset of Q]

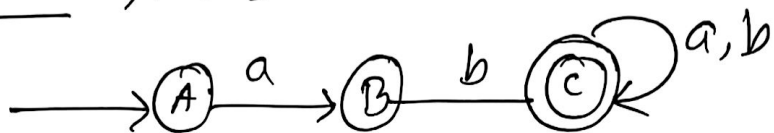
Example: starts with 'a'; $\Sigma = \{a, b\}$



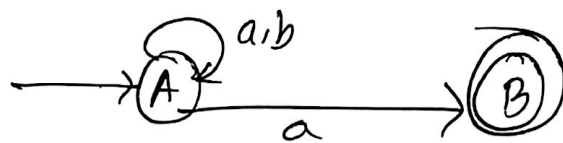
Here $\{A\}$ $\{A, B\}$ has not formed a relation with any other. This is that NFA describes.

* formal definition বলাই এই পড়ের দ্বারাওই নিহিত হবে।

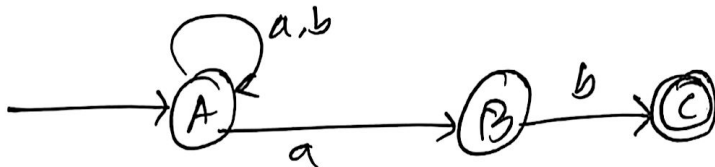
Example-2 starts with 'ab'



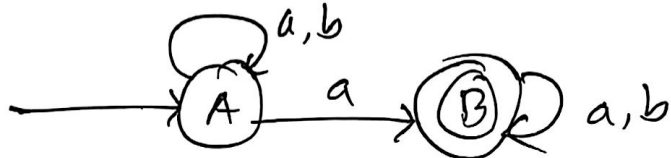
Example-3: ends with 'a'



Example-4: ends with 'ab'



Example-5: contains 'a'



Example-6: contains 'ab'

